GRADUATION PROJECT

Egyptian Criminal Justice Decision Support System

AI-Powered Multi-Agent Architecture

Using Graph RAG + Neo4j Knowledge Representation

2026

Table of Contents

Executive Summary

This graduation project presents a comprehensive AI-powered Decision Support System designed specifically for the Egyptian criminal justice system. The system leverages cutting-edge technologies including Multi-Agent Architecture, Graph RAG (Retrieval-Augmented Generation), and Neo4j knowledge graphs to provide structured legal analysis and decision support for criminal cases.

Key Features:

- Strictly adherent to Egyptian criminal law and Court of Cassation principles
- Multi-agent orchestration for specialized legal tasks
- Knowledge graph-based fact representation with full provenance tracking
- Procedural compliance auditing and evidence analysis
- Comprehensive citation and audit trail for all conclusions
- Explicit uncertainty flagging and reasoning transparency

1. System Overview

1.1 System Purpose

The system serves as a judicial decision support tool for Egyptian criminal cases. It is designed as an assistive tool for judges, not a replacement. The system provides structured analysis, legal research, and reasoned recommendations while maintaining complete transparency in its reasoning process.

1.2 Core Principles

- **Formal Legal Approach:** All analysis follows strict Egyptian legal formalism
- **Knowledge Sources:** Egyptian laws, Court of Cassation rulings, Egyptian judicial precedents, and case documents only
- **No External Sources:** Complete prohibition of non-Egyptian sources or general LLM knowledge outside the system graph
- **Transparency:** All ambiguities and uncertainties are explicitly stated in every section

1.3 Technology Stack

Component	Technology
Knowledge Representation	Graph RAG + Neo4j
Agent Framework	LangGraph Multi-Agent System
LLM Backend	Large Language Model (GPT-4, Claude, or equivalent)
Document Processing	OCR, NLP, Text Chunking
Audit & Logging	WORM Storage + Neo4j Provenance

2. System Architecture

2.1 Architectural Layers

The system is organized into five distinct layers, each with specific responsibilities:

1. **Ingestion & Preprocessing Layer:** Handles document intake, OCR, and initial structuring
2. **Knowledge Graph Layer (Neo4j):** Stores structured legal knowledge and case facts
3. **Multi-Agent Orchestrator:** Coordinates specialized agents and workflow
4. **LLM Reasoning Layer:** Performs constrained legal reasoning on structured data
5. **Audit / Logging / UI Layer:** Provides traceability, logging, and user interface

2.2 Agent Communication Protocol

All inter-agent communication follows a strict protocol to ensure traceability and prevent hallucination:

- **Immutable References:** All claims must reference Neo4j node IDs
- **Structured Messages:** JSON-serialized messages containing reference lists (no free-text summaries as sole references)
- **Provenance Logging:** Complete audit trail stored in Neo4j and WORM (Write-Once-Read-Many) logs

3. Multi-Agent System

3.1 Orchestrator Agent

3.1.1 Responsibilities

- Receives case files (facts, investigations, evidence, reports, precedents, defense documents)
- Distributes tasks to agents according to predefined workflow
- Aggregates intermediate outputs
- Manages case state lifecycle
- Prevents agents from operating outside authorized context

3.1.2 Global Context Management

The Orchestrator maintains a Global Context containing:

- Case Identifier
- Current charges
- List of input documents
- List of disputed issues
- Snapshot hash of latest graph state

3.1.3 Outputs

- Task dispatch plan
- Approved context windows for each agent
- Aggregated final reports for Judge Agent review

3.2 Data Ingestion & Graph Builder Agent

3.2.1 Core Rules

- **Converts unstructured text into legal knowledge graph**
- **No interpretation, no weighing, no inference**

3.2.2 Input Processing

Accepts: PDF/Text files including referral orders, police reports, forensic reports, prosecution investigations, evidence, previous rulings

Processing Steps:

  - OCR and text chunking
  - Extract facts, procedures, evidence, persons, and charges
  - Convert to Neo4j nodes and relationships
  - Link each input with document ID, page number, and text span

3.2.3 Outputs

- Executed Cypher queries
- Updated Neo4j graph database
- Provenance metadata for all extracted information

3.3 Procedural Auditor Agent

3.3.1 Core Rules

- Reviews procedural compliance according to Egyptian Criminal Procedure Code only
- Applies Court of Cassation principles on nullity (relative vs. absolute)

3.3.2 Inputs

- Arrest, search, and interrogation procedures
- Articles from Criminal Procedure Code
- Procedural principles from Court of Cassation

3.3.3 Processing Logic

- Matches each procedure against its legal requirements
- Determines procedure validity
- Identifies procedural nullity (if applicable)
- Assesses impact on evidence (fruit of poisonous tree doctrine)

3.3.4 Output Format

Procedural Audit Report (JSON structure):

```
{   "procedure": "تفتيش (Search)",   "status": "باطل (Null)",
"nullity_type": "بطلان مطلق (Absolute Nullity)",   "affected_evidence":
["E12", "E15"],   "legal_basis": ["Article 45 - Procedure Code", "Cassation
12/3/1998"] }
```

3.4 Legal Researcher Agent

3.4.1 Core Rules

- **Disciplined legal research only**
- **No fact analysis, no evidence weighing**

3.4.2 Inputs

- Charges from knowledge graph
- Keywords from case facts
- Extracted crime elements

3.4.3 Processing (Graph RAG)

- Vector search on legal articles and principles
- Cypher expansion: articles → principles → precedents
- **Automatic exclusion of all non-Egyptian sources**

3.4.4 Output Format

Legal Reference Package (JSON structure):

```
{   "charge": "ضرب أفضى إلى موت (Battery resulting in death)",   "statutes":
["Article 236 - Penal Code"],   "principles": ["The criterion is proving
criminal intent..."],   "precedents": ["Cassation 5/6/2003", "Cassation
14/2/1999"] }
```

3.5 Evidence Analyst Agent

3.5.1 Core Rules

- Analyzes probative value without final judgment
- **Does not issue verdicts, only assesses strength**

3.5.2 Inputs

- Evidence nodes, witness nodes, fact nodes from graph

3.5.3 Processing Logic

Linking:

- Physical evidence ↔ Facts
- Testimony ↔ Facts

Evaluation:

- Internal consistency
- Corroboration across sources
- Contradictions

Separation:

- Supporting evidence
- Contradicting evidence

3.5.4 Output Format

Evidence Strength Matrix (JSON structure):

```
{   "fact": "واقعة الاعتداء (Assault incident)",   "supports":
[     {"evidence": "سلاح (Weapon)", "strength": 0.85},     {"evidence": "شهادة
X (Witness X)", "strength": 0.7}   ],   "contradictions": [     {"evidence":
"شهادة Y (Witness Y)", "issue": "تناقض زمني (Temporal contradiction)"}   ] }
```

3.6 Defense Analysis Agent

3.6.1 Core Rules

- Analyzes defendant's defenses (formal and substantive)
- **No fabrication of defenses not present in case**

3.6.2 Inputs

- Graph subset: defenses, procedures, cassation principles
- Defense documents submitted to case

3.6.3 Processing Logic

- **Defense Classification:**
  - Procedural nullity
  - Insufficient evidence
  - Internal contradictions
  - Absence of criminal element
- Links each defense to its legal basis

3.6.4 Output Format

Structured Defense Argument (JSON structure):

```
{   "formal_defenses": ["بطلان تفتيش (Nullity of search)"],
"substantive_defenses": ["انتفاء القصد الجنائي (Absence of criminal
intent)"],   "supporting_principles": ["Cassation 21/1/1982"] }
```

3.7 The Judge Agent

3.7.1 Core Rules

- **Balancing only - no judgment without reasoning**
- **No weighing without legal basis**

3.7.2 Inputs

- Procedural auditor report
- Legal research findings
- Evidence analysis
- Defense arguments

3.7.3 Reasoning Process

6. Present established facts only (facts supported by procedurally sound evidence)
7. Present positions of parties (prosecution view based on admitted evidence, defense formal and substantive arguments)
8. Legal discussion (procedural validity, evidence sufficiency, statutory application)
9. Rely exclusively on Court of Cassation principles
10. Reach suggested conclusion (conviction / acquittal / lack of jurisdiction)

3.7.4 Uncertainty Handling

The Judge Agent includes explicit reservations when:

- Doubt exists
- Element is missing
- Evidence conflicts

Important Principle: *Doubt is always interpreted in favor of the defendant (hardcoded principle, no discretion)*

3.7.5 Outputs

- **Reasoned draft judgment**
  - Conviction
  - Acquittal
  - Lack of jurisdiction
  - Insufficient evidence
- **Explicit declaration of any deficiencies, doubts, or gaps**

4. Data Processing Pipeline

4.1 Data Cleaning & Normalization

- **Deduplication:** Remove duplicate entries
- **Date Unification:** Convert between Hijri and Gregorian calendars to single format
- **Fact-Opinion Separation:** Isolate facts from opinions, legal characterizations, and inferences (isolated content is not fed to graph)

4.2 Knowledge Graph Conversion

Graph Structure Principles:

- Each independent fact = Fact Node
- Each piece of evidence = Evidence Node
- Each procedure = Procedure Node
- Each legal article = LegalArticle Node
- Each cassation principle = CassationPrinciple Node

Critical Information Flow Restriction:

- ❌ LLM does NOT see original text documents
- ✅ LLM sees ONLY: Nodes + Properties + Relationships + IDs

4.3 Graph RAG Using Neo4j

Retrieval mechanisms:

- Cypher Queries for precise graph traversal
- Subgraph Expansion for contextual information
- Vector similarity search for semantic matching

The LLM only sees query results, never raw documents

4.4 Merging Cypher and Reasoning

Processing Flow:

11. **Query:** User question or analysis request
12. **Legal Subgraph:** Retrieved via Cypher queries
13. **Structured Context:** JSON/Schema formatted data
14. **LLM Reasoning:** Constrained inference on structured data
15. **Output:** Reasoned conclusion with full citations

4.5 Hallucination Prevention

Critical Safeguards:

- **Disable Free-Text Generation:** All outputs must be structured and referenced
- **Citation Required:** Every inference must cite source node IDs
- **Missing Data Handling:** Any missing data triggers explicit warning and suspension of judgment

5. Knowledge Graph Schema

5.1 Node Types

Node Type	Description
Case	Root node representing the entire criminal case
Fact	Individual factual claim or event
Charge	Criminal charge against defendant
LegalArticle	Article from Egyptian Penal Code or other statutes
Evidence	Physical evidence, testimony, or documentation
Procedure	Procedural step (arrest, search, interrogation, etc.)
DefenseArgument	Defense claim or argument
CassationPrinciple	Principle established by Court of Cassation
Judgment	Final or intermediate judgment/ruling

5.2 Relationship Types

Relationship	Meaning
`(Case)-[:HAS_FACT]->(Fact)`	Case contains this fact
`(Case)-[:HAS_CHARGE]->(Charge)`	Case includes this charge
`(Charge)-[:APPLIES_TO]->(LegalArticle)`	Charge based on this legal article
`(Evidence)-[:PROVES]->(Fact)`	Evidence supports/proves this fact
`(Evidence)-[:NEGATES]->(Fact)`	Evidence contradicts this fact
`(Procedure)-[:AFFECTS]->(Evidence)`	Procedure impacts evidence validity
`(Procedure)-[:DEFECTIVE_BY]->(CassationPrinciple)`	Procedure violates this principle
`(DefenseArgument)-[:REFUTES]->(Evidence)`	Defense challenges this evidence
`(DefenseArgument)-[:BASED_ON]->(CassationPrinciple)`	Defense cites this principle
`(Judgment)-[:BASED_ON]->(CassationPrinciple)`	Judgment relies on this principle
`(Judgment)-[:EVALUATES]->(Evidence)`	Judgment weighs this evidence

6. Ethical and Legal Constraints

The system operates under strict ethical and legal guidelines:

16. **Source Restriction:** Absolute prohibition on any non-Egyptian sources
17. **Jurisdictional Limitation:** Prohibition on any legal branch other than Egyptian criminal law
18. **Evidence Requirements:** No definitive judgments without complete criminal elements and lawful evidence
19. **Knowledge Boundary:** No general knowledge outside the knowledge graph
20. **Transparency Mandate:** All limitations, uncertainties, and gaps must be explicitly stated
21. **Presumption of Innocence:** Doubt is always interpreted in favor of the defendant
22. **Human Oversight:** System is an assistive tool only - final decisions remain with human judges

7. Implementation Checklist

Working Pipeline Development Steps:

Step	Task Description
1	Define complete Graph Schema with all node types and relationships
2	Setup and configure Neo4j database with appropriate indexes
3	Build Ingestion Pipeline (PDF → OCR → Nodes → Graph)
4	Implement LangGraph Agents with proper communication protocols
5	Configure Guardrails and validation rules
6	Implement Audit & Logging infrastructure
7	Test system on real Court of Cassation cases

8. Technical Specifications

8.1 System Requirements

8.1.1 Infrastructure

- **Database:** Neo4j Enterprise Edition (minimum version 5.0)
- **Agent Framework:** LangGraph for multi-agent orchestration
- **LLM Backend:** Compatible with OpenAI API, Anthropic Claude, or self-hosted models
- **Vector Search:** Neo4j native vector index or separate vector database (Pinecone, Weaviate)

8.1.2 Security & Compliance

- **Data Privacy:** End-to-end encryption for case data
- **Audit Logs:** WORM (Write-Once-Read-Many) storage for complete audit trail
- **Access Control:** Role-based access control (RBAC) for different user types
- **Data Residency:** All data must remain within Egyptian jurisdiction

8.2 Performance Metrics

- **Processing Time:** Complete case analysis within 15-30 minutes for standard cases
- **Accuracy Target:** 95%+ procedural compliance detection rate
- **Citation Coverage:** 100% of conclusions must include source citations
- **Hallucination Rate:** Target <1% through strict graph-based retrieval

9. Future Enhancements

9.1 Planned Features

- **Multi-Language Support:** Enhanced Arabic NLP capabilities and potential English interface
- **Case Similarity Search:** Find similar historical cases based on fact patterns
- **Predictive Analytics:** Estimate likely outcomes based on historical data (with appropriate disclaimers)
- **Integration with Court Systems:** Direct integration with existing case management systems
- **Continuous Learning:** Automated incorporation of new Court of Cassation rulings

9.2 Research Directions

- **Explainable AI:** Enhanced visualization of reasoning chains
- **Bias Detection:** Automated detection of potential biases in reasoning
- **Multi-Modal Evidence:** Integration of audio, video, and image evidence analysis
- **Cross-Jurisdictional Learning:** Comparative analysis with other civil law systems (while maintaining Egyptian primacy)

10. Conclusion

This graduation project represents a significant advancement in the application of artificial intelligence to the Egyptian criminal justice system. By combining Graph RAG technology, multi-agent architecture, and strict adherence to Egyptian legal principles, the system provides a robust, transparent, and accountable decision support tool.

The system's design prioritizes:

- **Legal Fidelity:** Strict compliance with Egyptian criminal law and procedures
- **Transparency:** Complete provenance tracking and explicit uncertainty declaration
- **Accountability:** Full audit trails and citation requirements
- **Human Oversight:** Recognition of the system as an assistive tool, not a replacement for judicial judgment

Through careful implementation of the outlined architecture, testing with real cassation cases, and continuous refinement based on judicial feedback, this system has the potential to enhance the efficiency, consistency, and quality of criminal justice decision-making in Egypt while maintaining the essential human element of judicial wisdom and discretion.

— End of Document —